AI Gateway — Five-Month Technical Report

Arvan Cloud AI Gateway · May–September 2026 · Extensive Technical Report

This report consolidates five months of design, benchmarking, and experimental work on the AI gateway. Every number is taken directly from the source documents (see `sources/`), which are renamed copies of the original reports. Confidence intervals (CI95) are paired bootstrap intervals over 20,000 resamples unless stated otherwise.

Timeline of the work:

PHASE	MONTHS	WHAT HAPPENED	KEY SOURCES
Foundations	Month 1	Technology stack, evaluation metrics, routing survey, baselines	01–04
Research & design	Month 2	55-technique verified inventory; 3-phase architecture	05, 06
Routing experiments	Month 3	RouterBench 3-router study, then nine-method comparison	07, 08
Routing synthesis + real pool	Month 3–4	Five-study synthesis; ArvanCloud 7-model pool test	09, 10
Compression & caching	Month 4–5	PCM ablations (Aug, Sep), scenario test, cache benchmark, live trial	11–16

Part I — Foundations (Months 1)

1. Technology stack selection

Source: `sources/01-core-technology-stack.pdf`.

The recommended stack, selected for AI-ecosystem compatibility, async throughput, and production readiness:

- **Backend:** Python + FastAPI (asynchronous request handling, low latency, automatic API docs)
- **LLM abstraction:** LiteLLM (unified interface across commercial and self-hosted providers)
- **Routing engine:** custom quality–cost–latency optimization engine (the project's core contribution)
- **Cache:** Redis (response cache, semantic-cache indexes, rate limiting, session data)
- **Database:** PostgreSQL (routing decisions, billing, quotas, historical performance; extensible with ClickHouse if analytics dominate)
- **Monitoring:** Prometheus + Grafana; **Logging:** OpenSearch/ELK
- **Deployment:** Docker + Kubernetes (horizontal autoscaling)
- **Local model serving:** vLLM

Go and Java were evaluated (concurrency / enterprise fit) but rejected in favour of Python's ML ecosystem; Flask and custom SDKs were rejected for scalability and maintenance reasons respectively.

2. Evaluation metrics reference

Source: `sources/02-evaluation-metrics-reference.pdf`. Defines the dashboard vocabulary used throughout all later studies:

- **Quality:** Win Rate (routed output vs premium baseline), Pref Accuracy (user-preference match), Judge Score (1–5 by advanced models), Task Metrics (F1, ROUGE, Exact Match).
- **Cost:** Cost Savings (% reduction vs premium baseline), Norm. Cost ($/1K output tokens), Router Overhead (monetary cost of running the router itself).
- **Latency:** TTFT (time to first token, ms), TPOT (ms per output token), Router Latency (decision time, ms).
- **Trade-off:** AUCQ (area under the cost-quality curve), Cost @ X% quality, Pareto frontier (set of configurations where no metric improves without degrading another).

3. Advanced routing methodologies survey

Source: `sources/03-advanced-routing-methodologies-survey.pdf`. Catalogued eleven methodologies with mechanism, complexity, Pareto impact, and data prerequisites: Router-BERT (classification; ~70% cost reduction claim), Matrix Factorization (latent embeddings from sparse score matrices), Entropy-Based Cascade (real-time uncertainty escalation; needs self-hosted logits), Dynamic Thresholding/SWOR (budget- and load-adaptive thresholds), Intent-Driven Anchoring (cosine similarity to pre-computed vector anchors; <5 ms decisions), Token-Budget Routing (output-length-aware decode-cost optimization), Speculative Verification (local draft + cloud verification; strict tokenizer alignment), Multi-Agent Meta-Routing (domain micro-routers under a meta-router), Reward-Model Guided routing (edge RM predicting routing reward), KNN Historical Routing (vector-DB nearest successful decisions), Multi-Armed Bandit (explore/exploit model selection with online learning). This survey became the menu the later RouterBench studies ordered from.

4. Baselines

Source: `sources/04-routing-baselines.pdf`. Reference points that any advanced router must beat:

- **Naive:** uniform random routing (cost/latency-constrained), round robin, static extremes (always cheapest / fastest / best).
- **Heuristic:** prompt-length routing (short → fast model, long → capable model); keyword routing (reasoning/math/code keywords → strong model).
- **Greedy utility:** $U_i = w_1 Q_i − w_2 C_i − w_3 L_i$ under constraints $C_i ≤ C_{max}$, $L_i ≤ L_{max}$ (Cost-of-Pass-inspired).
- **Cascade routing:** cheap model first, lightweight evaluator escalates if the response is insufficient (FrugalGPT-style).

Part II — Verified technique inventory and architecture (Month 2)

5. The verified technique inventory

Source: `sources/05-verified-technique-inventory.pdf` (35 pages; 55 techniques across 7 areas; produced by 7 parallel research agents and corrected by 7 adversarial fact-check agents against primary sources; labels: evidence-based / reasoned-estimate / unverified-assumption).

Highlights that shaped later decisions (all numbers verified against the named papers):

Routing. Arch-Router-1.5B: 93.17% routing-decision accuracy, 98.11 fine-grained query-to-policy matching, +7.71% average over top proprietary routers, 51±12 ms latency, >28× faster than Claude-Sonnet-3.7 (1450±385 ms) — measured on the paper's own curated benchmark. RouteLLM: −85% cost on MT-Bench / −45% MMLU / −35% GSM8K while retaining 95% of GPT-4 quality; MF variant is microsecond-scale. Hybrid

LLM tri-metric prediction: up to 40% fewer calls to the large model with no measured quality drop. GraphRouter: minimum +12.28% on its joint Reward metric; UniRoute routes across 30+ unseen LLMs. RouterBench itself: 405,467 inference outcomes from 11 LLMs across 8 datasets (64 tasks + 1 RAG dataset); its own comparative analysis found none of the tested routing algorithms significantly outperformed the Zero-router baseline — which independently validated our rules-and-cascade-first ordering. The DSC robustness study (EACL 2026): a BERT router directed ALL coding/math queries to the strongest model and routed jailbreak attempts to weaker models — motivating router hardening.

Cascading & scoring. FrugalGPT: up to 98% cost reduction (HEADLINES 98.3%; OVERRULING 73.3%; COQA 59.2% — narrow classification tasks), with a DistilBERT regression scorer; best case +4% accuracy over GPT-4 at equal cost. Early-abstention cascades: −2.2% overall test loss, +4.1% abstention rate, −13.0% cost, −5.0% error rate (across GSM8K/MedMCQA/MMLU/TriviaQA/TruthfulQA/XSum). Dekoninck unified routing+cascading: +1..4% on RouterBench over the best pure strategy, but margins shrink to ≤1.2% with noisy quality estimators (up to 14% on SWE-Bench with accurate post-hoc estimates) — the edge collapses without calibrated estimators. Logprob quantile deferral: free on vLLM, ~0 ms; recalibration is a release gate on every model swap. Confidence-based deferral "works remarkably well" in-distribution; fails under specialist models, label noise, distribution shift.

Caching. vLLM automatic prefix caching: TTFT 4.3 s → 0.6 s on a repeated ~10K-token prompt (Qwen3-32B; best-case full-prefix rehit); +13.3% throughput measured by SqueezeBits (Llama-3.1-8B, 1×A100, 25% shared prefix); TPOT ~+9.8% faster. KV-aware scheduling across replicas: TTFT P90 0.542 s precise vs 92.6 s random in a deliberately cache-saturating stress benchmark (8 pods, 16×H100, 150 tenants × 6K-token contexts); throughput 8,730 vs 6,944 output tok/s (+25%) over approximate scheduling. Exact-match cache: −100% marginal cost per hit, <1 ms on misses; hit rate ~0–5% on freeform human chat. Semantic caching: MeanCache's 31% repeat-query anchor comes from a 20-user self-reported study; GPTCache produced 233 false hits vs MeanCache's 89 on 700 unique queries; GPT Semantic Cache reports 61.6–68.8% API-call reduction on repetition-heavy categorized traffic. vCache: up to 12.5× higher hit rate and 26× lower error at a user-defined error bound. CacheAttack: 86% average black-box response-hijack rate against a GPTCache-style cache; 80.6% on AWS Bedrock and 88.3% on Azure API Management production semantic caches — per-tenant isolation and verified-write gates are non-negotiable. Krites grey-zone promotion: up to 3.9× more requests served from the curated static tier (search-workload maximum; conversational ~2.4×).

Compression. Area verdict: no neural compression in the MVP. Non-neural router-path condensation (TextRank + TF-IDF, vLLM Semantic Router pattern): domain classification +8.1 pp (53.1%→61.2%), PII detection +13.9 pp (78.5%→92.4%), but jailbreak detection −14.2 pp (70.8%→56.6%) — condensed text must never feed safety classifiers; ~19 ms CPU at 16K-token input; the paper's 98× headline is mostly GPU+FlashAttention (38.7×), with condensation contributing 2.0×. LLMLingua-2: 2–5× compression, but in-the-wild replication finds only up to 18% e2e speedup, and only beyond ~5k-token prompts; on code "even slight compression harms model outputs" (practical ratio ceiling ~1.4× on LongBench LCC). LongLLMLingua: up to +21.4% accuracy on multi-document QA at ~4× fewer tokens; 94.0% cost reduction on LooGLE (best case). Multi-turn history summarization: −30..70% session tokens on >20-turn sessions by arithmetic, coherence risk concentrated in wrong-preservation failures. RECOMP abstractive: compression as low as 6% of original tokens, but training a compressor is the wrong effort profile for a 6-person team.

Parallel & ensemble. Self-consistency: GSM8K +17.9, AQuA +12.2, SVAMP +11.0, StrategyQA +6.4, ARC-c +3.9 points absolute (with ~40 sampled paths on 2022-era base models — treat as ceiling). Verifier-gated best-of-N: SWE-bench Lite 15.9% → 56% at 250 samples (coverage with an oracle verifier); Reflexion with external test signals reaches 91% vs 80% pass@1 on HumanEval; intrinsic self-correction on reasoning is ~0 to negative (Huang et al. 2023) — "never refine without a verifier." MoA (open-source only): AlpacaEval 2.0 LC win rate 65.1% vs GPT-4o's 57.5% at 6 proposers × 3 layers (~10–18 calls). Multi-model debate: EXCLUDED — dominated by self-consistency at equal compute (Smit et al., ICML 2024). Hedged dispatch: Dean & Barroso's 99.9th-percentile 1800 ms → 74 ms at ~2% extra requests is a 100-server BigTable read fan-out — an analogy, not an LLM figure; +2..5% cost is the transferable part.

Scheduling & serving. Budget ledger + watchdog is the defining capability. Output- length prediction: S3's DistilBERT 98.61% bucket accuracy at ~3.7 ms (10 equal-width buckets); SSJF −30.5..39.6% average JCT vs FCFS. Retries with backoff+jitter and per-endpoint circuit breakers: existential for the sanctions-risk closed-API tier. QLM priority scheduling: +40–90% SLO attainment vs vanilla vLLM. Speculative decoding: 2–3×

faster decoding (Leviathan et al.); vLLM measured up to 2.8× at low QPS; decays to ~1× at saturation. P/D disaggregation (DistServe 7.4×, Splitwise 1.4–2.35×, Mooncake up to 5.25×): premature below two-digit GPU counts. Quantization: FP8 W8A8 effectively lossless; INT4 W4A16 ~99% score recovery over 500k+ evaluations on the Llama-3.1 family; W4A16 most cost-efficient for synchronous setups, W8A8 dominates in asynchronous continuous batching; cheap-tier serving cost −40..70%.

Quality measurement. GPT-4-class judges: 85% agreement with humans on MT-Bench (human–human 81%); Prometheus-2-7B Pearson 0.882 in-domain but 0.54–0.69 out-of-domain; expect 0.55–0.65 realistic out-of-distribution. Top 8B reward models: RewardBench v1 97.8 (saturated), v2 86.5; Skywork-Reward-V2-Qwen3-0.6B nearly matches a prior 27B RM (70.9 vs 71.6 across 7 benchmarks). promptfoo golden set of 200–500 requests detects ≥5 pp regressions. Template-first XAI is faithful by construction; free-form LLM rationales are documented to misstate decisions (Turpin et al.).

6. The 3-phase architecture

Source: `sources/06-architecture-and-technique-analysis.pdf` (v1.1, generated 2026-07-24; merged pragmatic spine + staged research ceiling). The one-sentence architecture, quoted from the source:

> Rules plus a 1.5B policy router pick the cheapest feasible model in under 65 ms; a ledger that every call debits before dispatch and a watchdog that fires at deadline − t_floor make budget violations structurally impossible; free signals (verifiers, logprobs, caches) decide when to spend more — and when they do, leftover budget buys thinking tokens on a reasoning tier before it buys hand-built orchestration; every answer passes an output guard before it leaves; and every clever component from the research ceiling sits behind a feature flag whose enable condition is a measurement, not an opinion.

Key structural elements:

- **Phase 1 (input analysis & routing):** auth + budget ledger init + global deadline watchdog; PII redaction with deterministic session-stable placeholders + injection/jailbreak guard on full raw text; exact-cache lookup (Redis, <1 ms); raced semantic cache (Stage-2); context canonicalization (static prefix first, byte-stable); non-neural condensation above L_condense = 2048 tokens (classifier input only, ≤20 ms); classification via config rules (<1 ms) + Arch-Router-1.5B (51±12 ms, hard 75 ms timeout); tri-metric prediction (cost = price × length bucket × margin; latency = p95 TTFT + len × p95 TPOT; quality = policy × model golden-set table); feasibility filter; degradation ladder.
- **Phase 2 (execution strategy), ordered first-match:** semantic-hit serve; MoA (flag OFF); reasoning-tier thinking-budget dispatch; Best-of-N; sequential cascade; speculative parallel cascade; single-shot default. Execution mechanics: pre-spend reservation on the ledger, per-call timeout = min(T_rem − t_floor, p99 cap), health/prefix-affinity endpoint pick, TTFT-based hedging (max 1, +2–5% cost cap), max 1 retry with alternate endpoint, active cancellation discipline.
- **Phase 3 (result synthesis & validation):** tier-0 deterministic verifiers first; logprob quantile confidence; RM scorer (Stage-3); gated in-path judge; acceptance decision; multi-candidate selection; refine loop (never without a verifier); deadline-aware escalation guard; output guard (regex floor at MVP, classifier layer Stage-2, rolling-window holdback for streams, distinct output-policy-violation error class); assembly with un-redaction; template XAI <1 ms; cache-write gate (verified, post-redaction, tenant-namespaced); accounting close.
- **Feature-flag ladder:** every non-core optimization is a flag with a measured enable condition and an auto-degrade/kill condition (e.g. exec.cascade: enable at savings ≥40% with <5% golden-set regression; cache.semantic: hit rate ≥15% at observed error ≤ ε; exec.bon: golden uplift ≥ +5 pp on hard votable slice AND GPU util <70%). The rules-only router floor is never removed.
- **Assumption register (A1–A15):** launch traffic assumed mostly short (<2k tokens), single-turn, easy-skewed, Persian/English mixed; multi-tenant from day one; closed-API dispatch disabled by

default per tenant; no human-annotation budget; ~1 replica per model. Each assumption carries the question that resolves it.

Part III — Routing experiments (Months 3–4)

7. RouterBench, first study: three routers

Source: `sources/07-routerbench-three-routers-report.pdf` (Persian-language report).

Motivation. The pre-existing baseline was an LLM judge picking the model per prompt: 61.25% quality at $0.000223 per query on 100 random prompts. The problem was not the quality number but interpretability: it had never been compared against the three references that give it meaning — oracle (best model per prompt), best constant, and random. Computed on the same 100 prompts: always-Yi-34B reached 65.00% at $0.000176 — both better quality and cheaper than the judge. The judge picked a single model on 91 of 100 rows; its deviations on the other 9 cost 33.3% vs 22.2% for the constant (n=100, CI95 [−13.75, +6.25] — spans zero; the correct claim is that the experiment could not show the judge's superiority, not that it was proven worse).

Data fixes found along the way: a preprocessing bug passed Python `str()` / `repr()` strings to the judge (extra quotes/brackets); a newline had dropped `claude-v1` from the 11-model candidate list, leaving 10.

Selection. Of the eleven methodologies in the survey (source 03), three met RouterBench's data prerequisites and were implemented: Matrix Factorization (#2), Intent-Driven Anchoring (#5), KNN Historical (#10). Dynamic Thresholding (#4) was partially implemented (λ sweep mechanism, not an independent router). Entropy-Based Cascade, Token-Budget, and Speculative Verification were impossible on RouterBench data; Router-BERT, Multi-Agent Meta-Routing, Reward-Model, and Bandit were not run at this stage.

Architecture. All routers share one interface — `fit(train)`, `predict(prompts)` returning predicted quality and cost per candidate — and none makes the selection itself. Selection is a shared utility function, U = Q − λ·C, swept over twenty λ values (λ=0 is "quality at any price"; large λ is "cheapest"). This keeps differences attributable to prediction quality, not selection policy.

- **KNN Historical (#10):** training prompts hashed TF-IDF (2,048-dim); for a new prompt, 32 nearest neighbours by cosine similarity; weighted average of their scores/costs. Pure lookup; no real training.
- **Intent-Driven Anchoring (#5):** spherical k-means with 128 anchors over the training set; each anchor stores the precomputed mean profile of the 11 models. Routing = nearest anchor — a matrix multiplication independent of corpus size. Clusters smaller than three members shrink toward the global mean (noise protection).
- **Matrix Factorization (#2):** RouterBench has no repeated prompts, so classic prompt×model factorization cannot be fit. Adaptation: embedding as a linear function of features, predicting the score matrix with reduced-rank ridge regression (closed-form; log10 of prompt length among features; cost regressed as well).

Results (7,296 held-out prompts; best constant = always gpt-4 at 78.11% and $0.003270/query; oracle ceiling 91.43%):

ROUTER	QUALITY	$/QUERY	SAVINGS VS GPT-4	Δ QUALITY (CI95)
#5 anchor	**78.67%**	**$0.002751**	**16%**	**+0.57 pp [+0.22, +0.91]**
#10 knn	77.81%	$0.002483	24%	−0.29 pp [−0.75, +0.16]
#2 matrixfact	76.55%	$0.002895	11%	−1.56 pp [−2.03, −1.10]

Anchor is the only method that is simultaneously better and cheaper than the floor with an interval excluding zero — a significant Pareto-frontier improvement. KNN wins on cost (24% cheaper) but its quality interval

spans zero. Matrix factorization is significantly worse (its structural limitation: no unique repeated prompts to factor). On the full Pareto frontier, anchor covers almost all non-dominated points: ~74.20% quality at $0.000551/query — about 83% cheaper than gpt-4 for ~3.9 points of quality.

Versus the LLM-judge baseline (same 100 judge-routed prompts, 36,381 remaining prompts fit):

METHOD	QUALITY	$/QUERY	SAVINGS	DECISION TIME
LLM-as-judge (baseline)	61.25%	$0.000223	—	~2180 ms
matrixfact	69.25%	$0.000203	9%	~0.4 ms
knn	67.75%	$0.000154	31%	~0.4 ms
anchor	64.75%	$0.000143	36%	~0.4 ms

All three are better, cheaper, and about three orders of magnitude faster. Against the design budget of routing overhead p99 ≤ 25 ms, the judge overshoots ~90×; the learned routers stay ~60× under it. Fairness note: the routers had 29,185 labelled training samples the judge never saw; the correct argument is that we possess this data and a method using it beats an LLM call on the critical path at a fraction of the latency.

Self-hosted latency measurement. RouterBench's `predicted_total_latency_ms` field had no backing, so TTFT and TPOT were measured directly (60 samples per model, one 20-GB GPU, 5 prompt lengths × 2 generation lengths):

MODEL	QUANTIZATION	TTFT (FIT)	R^2	TPOT	TOTAL @ FULL
Mistral-7B	FP16	55.9 + 0.1525·tok	0.90	43.6 ms	11,375 ms
WizardLM-13B	AWQ INT4	53.3 + 0.3100·tok	1.00	23.1 ms	6,106 ms

The 13B INT4 model decodes ~2× faster than the 7B FP16 model, but prefill is ~2× slower — the two phases are bound by different resources (decode by memory bandwidth, prefill by parameter volume) — direct empirical support for the local-tier quantization decision. Both measurements run at ~90% of theoretical memory-bandwidth ceiling (92% and 88%), i.e. physics-consistent. OpenAI/Anthropic models (need API keys) and Mixtral/Llama-2-70B (do not fit a 20-GB card) were deliberately not estimated: recorded as `measured: false`.

8. RouterBench, full study: nine methods

Source: `sources/08-routerbench-nine-methods-report.pdf` (Persian-language report).

Setup. 36,481 questions, 11 models with recorded scores and dollar costs; a router sees only the question text and picks a model, scored from the table. Split: 29,185 train / 7,296 held-out (all numbers on held-out). Model pool (mean score, $/query): gpt-4-1106-preview 78.11% / $0.003270 (best, priciest); Yi-34B-Chat 65.37% / $0.000186 (best score:cost); claude-v2 63.60% / $0.002415; claude-v1 62.74% / $0.002130; gpt-3.5-turbo-1106 62.29% / $0.000243; claude-instant-v1 60.26% / $0.000233; mixtral-8x7b-chat 54.78% / $0.000134; WizardLM-13B 43.71% / $0.000073; llama-2-70b-chat 33.80% / $0.000203; mistral-7b-chat 30.40% / $0.000046 (cheapest); code-llama-instruct-34b 20.11% / $0.000172. Benchmark mix: MMLU 38.5% (partial-credit 0/¼/½/¾/1), HellaSwag 27.5%, GSM8K 20.4%, ARC/WinoGrande/MBPP 8.7%, Chinese subsets 2.4%, MT-Bench 0.3% (0.1–1 judge).

References. Oracle (cheapest correct per question): 91.43% at $0.000230. Best constant (always gpt-4): 78.11% at $0.003270. Random: 51.75% at $0.000849. Notably, in 82% of questions some cheaper model answers correctly, and the oracle is ~14× cheaper than the floor — the money is in the gap between them.

Results (all cost-matched to gpt-4 spend; CI95 paired bootstrap):

#	METHOD	SCORE	$/QUERY	SAVINGS	Δ VS FLOOR (CI95)	DECISION TIME
1	Router-BERT	78.74%	$0.002476	24%	+0.63 [+0.24, +1.03]	15.2 ms (GPU)
6	Token-Budget	78.70%	$0.002708	17%	+0.59 [+0.24, +0.94]	1.55 ms (anchor +)

#	METHOD	SCORE	$/QUERY	SAVINGS	Δ VS FLOOR (CI95)	DECISION TIME
5	Anchor	78.67%	$0.002751	16%	+0.57 [+0.22, +0.91]	0.64 ms
8	Meta-Routing	78.61%	$0.002782	15%	+0.50 [+0.19, +0.81]	1.60 ms
10	KNN	77.81%	$0.002483	24%	−0.29 [−0.75, +0.16]	17.9 ms
2	Matrix Factorization	76.55%	$0.002895	11%	−1.56 [−2.03, −1.10]	0.72 ms
9	Reward-Model	64.71%	$0.002151	34%	−13.40 [−14.41, −12.40]	1.46 ms
11	Bandit (online)	79.40%	$0.002723	17%	— (different method)	~0.7 ms
4	Dynamic Threshold (budgeted)	78.36%	$0.001635	50%	— (different method)	~0.7 ms

Notes from the source: Bandit learns online (its 79.40% is on the test stream after warm-up; its small edge over anchor comes from feedback the other routers never see; cold-start without anchor history costs ~12 points over the stream). Dynamic Threshold works at a fixed budget, re-tuning the dial after each question from remaining money — 78.36% at half of gpt-4's spend, final spend error within 0.2%. Reward-Model loses 13.40 points for an instructive reason: its reward model correlates +0.55 with answer length and only +0.03 with correctness, so it drifts toward verbose models — for gpt-4 the correlation is even negative.

Two negative results worth recording: Meta-Routing with six real scopes stayed within 0.05 points of global anchor (domain gating added nothing — RouterBench's question mix doesn't segment); combining BERT + anchor + KNN (weighted average, then gated) gave at best +0.10 points, within noise, and the learned weight for KNN was zero.

The two winners head-to-head (BERT vs anchor, same questions, seven budget levels, budget = spend relative to gpt-4): 5%: 67.28 vs 67.05 (−0.23 [−0.84, +0.39]); 10%: 70.61 vs 71.00 (+0.39 [−0.24, +1.04]); 20%: 75.23 vs 75.49 (+0.26 [−0.25, +0.78]); 30%: 77.48 vs 77.75 (+0.26 [−0.01, +0.54], borderline); 50%: 78.30 vs 78.34 (+0.04 [−0.29, +0.37]); 75%: 78.60 vs 78.80 (+0.19 [−0.08, +0.47]); 100%: 78.67 vs 78.83 (+0.15 [−0.12, +0.43]). Seven ties; BERT is ahead slightly at 70–78% but never crosses zero. Cost side: anchor is 1–6% cheaper at 76–78% quality; BERT is 10–13% cheaper to reach 70–74%. Decision: anchor — same quality, 24× faster (0.64 ms vs 15.2 ms), CPU numpy only, no GPU, no 10-minute fine-tune; GPU costs appear in no cost column of the study (those count only candidate-model spend). If operating a GPU router at a curve point where model spend exceeds ~10% GPU keeping costs, BERT's precision could matter.

9. Five-study synthesis

Source: `sources/09-routing-cascading-synthesis.pdf` (Gemini-prepared, source-grounded).

Executive takeaways (quoted/condensed): Anchor is the most practical routing solution — 24× faster than GPU methods (0.64 ms on CPU) while matching top accuracy. Cascading is frequently redundant over a strong standalone router: across 23 head-to-head comparisons at matched spend, standalone anchor beat anchor+cascade in every single one (+5.29 pp vs −1.59 pp vs coin flips), with cascades adding pure cost overhead (2.5× calls). The primary leverage point is escalation specificity.

Real list pricing vs token proxy (ArvanCloud pool; 150 MMLU-Pro questions, 90 held out, OpenRouter list prices 2026-09-22, 12 splits): prompt+4×completion token spans 4.9×, but real list prices span 86× and invert the cost ordering. Accuracy (±SD) and cost/query: Always GPT-5.6-Sol 93.1±1.3% / $0.001755; Anchor top-3+cascade 92.9±1.9% / $0.001338; Matrix-Factorization 92.0±2.2% / $0.001974; Anchor alone 90.4±2.5% / $0.000866; Always DeepSeek-V4-Flash 89.5±2.1% / $0.000106; Random choice 88.7±2.1% / $0.001432. Frontier reduces to two models (GPT-5.6-Sol and DeepSeek-V4-Flash).

Training-data density explains router performance: anchor beat weighted coin flips on RouterBench (+5.16 pp over the convex hull, winning at 98% of operating points) but lagged on ArvanCloud (−0.62 pp, winning at 7%): RouterBench provided 29,185 training prompts (128 anchors) vs 60 training prompts (8 anchors, 7 per cluster). Anchor learns which prompts are easy enough for a cheap model, not topic suitability.

Escalation-signal deconstruction (shortlist fixed, only the wrongness signal varied, vs anchor): agreement-then-weighted-vote −0.73 pp accuracy at 2.22× cost and 2.50 calls/query; agreement-then-anchor's-pick −1.58 pp at 1.66× and 2.26 calls; perfect wrongness signal +10.84 pp at 0.87× cost and 1.34 calls. Sensitivity P(escalate | wrong) buys accuracy (0.5→1.0 is worth +10.4 pp); specificity P(stay | right) buys cost (1.0→0.5 raises the bill from 0.88× to 1.60×). The agreement signal's sensitivity is 0.743 (at target) but specificity only 0.619 — it escalates 38% of correct answers for no benefit. Target: hold sensitivity ~0.75 and push specificity 0.62 → 0.75.

Caveats: OpenRouter list prices stood in for ArvanCloud tariffs; 90 held-out questions entail ~±6 pp variance; per-token logprobs were available on only 1 of 9 evaluated models (blocking BiLD-style confidence cascades).

10. ArvanCloud pool: routing and cascading

Source: `sources/10-arvancloud-routing-cascade-report.pdf` (Persian-language report).

Dataset. MMLU-Pro: 12,032 ten-choice questions in 14 domains; balanced sampling of 11 per domain produced 154, of which every 7th had all models answer — 150 common questions (60 train / 90 held-out), 14 domains, 10 options each. Answer format "The answer is (X)" — no judge in the loop. Example (business domain): a car insurance bought March 6 for $155, cancelled September 6 — correct refund $62 (option H); of 7 models, Opus, Sol, DS-Flash answered H; Sonnet, Terra, Flash-Lite, Luna answered J. Terra spent 981 output tokens and was still wrong; Sol spent 349 and was right — exactly where a router must not hand the cheapest model the question. Difficulty distribution across the 150: 105 questions (70%) answered by all 7 models; 41 (27%) headroom; 4 (3%) answered by none.

Results (90 held-out questions):

MODEL / POLICY	λ	ACCURACY	TOKENS (IN/OUT)	CALLS
GPT-5.6-Sol	—	92.7%	750 / 207	1.00
Claude-Opus-4.6	—	92.0%	1193 / 346	1.00
Claude-Sonnet-4.6	—	89.3%	974 / 300	1.00
DeepSeek-V4-Flash	—	88.7%	2148 / 945	1.00
GPT-5.6-Terra	—	86.0%	782 / 244	1.00
Gemini-3.1-Flash-Lite	—	86.0%	1391 / 315	1.00
GPT-5.6-Luna	—	82.0%	439 / 67	1.00
anchor	0	**93.3%**	**637**	1.00
anchor top-3 + cascade	0	94.4%	1505	2.07
anchor	≥.002	83.3%	428	1.00
anchor top-3 + cascade	≥.002	90.0%	1226	2.16
cascade (ETH)	any	83.3%	428	1.00

The four policies compared. Always-Sol (reference, 92.7%, 750 tokens, 1.00 call); anchor (decides from prompt text before seeing any answer: 93.3%, 637 tokens, 1.00 call); ETH cascade (per "A Unified Approach to Routing and Cascading for LLMs"; estimates accuracy, cost, and a confidence σ per model, deciding whether the next call is worth it — in practice never escalated: 83.3%, 428 tokens); anchor top-3 + cascade (anchor nominates top-3 candidates, the two cheaper run, agreement serves, disagreement runs the third and takes a quality-weighted vote: 94.4%, 1,505 tokens, 2.07 calls).

Why no logprob cascading: the common approach (BiLD) relies on token-by-token confidence of the cheap model, but the cheap endpoint — DeepSeek-V4-Pro — is the most expensive to probe; only 1 of 9 models returned logprobs. Agreement between models was the only logprob-free signal available.

Bottom line (from the source). Anchor alone beats all seven pool models: 93.3% accuracy for 637 tokens, versus the best pool model (Sol) at 92.7% and 750 tokens — deciding from the question text raises accuracy

and lowers cost, not one at the price of the other. Adding cascade buys +1.1 points for 2.4× tokens; on 90 questions the error interval runs 0 to 3.3 points, so a true zero effect cannot be excluded. It is the rational choice where accuracy per question dominates, but not the default.

Part IV — Prompt compression & context management (Months 4–5)

11. PCM ablation, August: does compression hurt routing?

Source: `sources/11-pcm-ablation-august.pdf` (2026-08-28; ~1,600 judge calls; judge gpt-5.1-nano @ temp 0; RouterBench ground truth 36,481 rows; 100 prompts per arm; 1 error).

Four-stage module tested against Sajjad's LLM-as-judge routing baseline, same judge, same dataset — only the prompt preprocessing changes.

Study 1 — long prompts (synthetic RAG, ~6,000 tokens per prompt; RouterBench has no long prompts, so questions were wrapped in retrieved context; compressor pointed at context only):

ARM	ACCURACY	Δ	95% CI	JUDGE TOKENS/CALL	RE-ROUTED
baseline	61.25%	—	—	6,307	—
gated (our default)	61.25%	+0.00 pp	[−4.0, +4.0]	3,303	20%
compress_r75	63.25%	+2.00 pp	[−2.0, +6.0]	4,886	23%
compress_r50	60.25%	−1.00 pp	[−7.0, +5.0]	3,194	23%
compress_r25	62.25%	+1.00 pp	[−2.0, +4.0]	1,861	25%
rewrite	63.13%	+2.02 pp	[−2.0, +6.1]	6,303	18%

Headline: on long prompts the shipping configuration compressed 95 of 100 prompts, cut judge input tokens 47.6%, and landed exactly on baseline accuracy (±0.00 pp). Judge noise floor ±1.50 pp, measured from 5 identical-input runs (which re-routed 2–7 of 100 prompts on their own). The honest claim is "quality does not measurably degrade," not "quality improved" — the two arms outside the noise band (+2 pp, in our favour) have CIs spanning zero.

Study 2 — cache reuse (cheapest win): the canonicalizer only reorders (stable content first, volatile last; identical bytes, identical token count). Canonical order: 28/30 calls hit the provider cache (76.1%; 57,600 of 75,652 tokens served from cache). Scrambled order: 0/30 (0.0%). A false start is recorded: the first run measured 0% for both — the judge prompt was 519 tokens, below the provider's cache minimum; below that floor ordering is irrelevant, above it ordering is everything.

Study 3 — short prompts (RouterBench as shipped): all 36,481 prompts are under 2,000 tokens (median 178, max 1,026); compression pays above ~5,000. The default declined all 100 prompts, correctly. Forced past its own gate: baseline 62.25% / 519 tokens / 2.03 s; gated 60.75% (−1.50) / 519 / 1.92 s; compress_r75 59.75% (−2.50) / 472 (−9%) / 2.32 s; r50 61.25% (−1.00) / 414 (−20%) / 2.15 s; r25 61.75% (−0.50) / 370 (−29%) / 2.51 s; rewrite 61.25% (−1.00) / 513 (−1%) / 2.24 s. Fewer tokens did not mean a faster judge — r25 sent the fewest tokens and was slowest; at ~500-token inputs latency is fixed overhead. Token saving is a third of the prompt-level figure because the judge also reads a fixed 330-token scaffold.

Short vs long side by side: median prompt 178 vs 5,990 tokens; gate fired 0/100 vs 95/100; our-default judge tokens 519 (0%) vs 3,303 (−47.6%); forced r50 414 (−20%) vs 3,194 (−49.4%); accuracy Δ −1.50 pp (declined all) vs +0.00 pp; re-routed 7% (=noise) vs 20%; PCM cost 41 ms vs ~640 ms; verdict NOT WORTH RUNNING vs WORTH RUNNING. The mechanism: forced r50 cuts ~55% of the prompt in both cases, but the fixed 330-token scaffold dominates short inputs — the measured basis of the 5,000-token gate. (The two 61.25% accuracies are a coincidence of different datasets, not a match.)

Stage verdicts: canonicalize — SHIP ON (0% → 76% cache tokens, 2.5 ms, cannot alter text); compress — KEEP THE GATE (−47.6% tokens on long prompts at zero measurable accuracy cost, ~640 ms, fires above

5,000 tokens); summarize — PROMISING (−81.4% session tokens on 33-message chats, question + recent four turns verbatim in 25/25, 3.5 s once per session, quality untested at this stage); rewrite — LEAVE OFF (rejected its own output on 80–90% of prompts; 1.1–1.4 s per prompt for nothing).

Two bugs found that 95 unit tests missed: (a) the code detector matched ordinary English (`from` , `return` , `class` , `public`) — any two in a paragraph disabled compression; it mislabelled 28 of 100 plain multiple-choice questions; now 0/100; (b) duplicate force-tokens crash LLMLingua — the same PII placeholder repeated fails with a bare AssertionError (survives 60 repeats, dies at 120, exactly the prompt size the stage exists for).

Honest-limits section (from the source): long-prompt and multi-turn sets are synthetic (distractors are unrelated RouterBench questions — the easy case; read −47.6% and −81.4% as upper bounds); n=100 cannot resolve these effect sizes (CIs ~±4 pp on ~1 pp differences); this measures routing decisions, not answer quality; downstream cost rose slightly on re-routing ($0.000257 vs $0.000217 per query — noise at this size, not a saving).

12. PCM ablation, September: the corrections round

Source: `sources/12-pcm-ablation-september.pdf` (2026-09-11; 1,000 questions per test; ~12,700 judge calls; 0 errors; judge gpt-5.1-nano).

Headline numbers: 82% text reused from cache; −48% text in long documents at −0.7 points of quality; −40.8 points if chats are summarized. Two methods ship (reorder, long-document shortening); one is dangerous (summarization); one is useless (rewrite).

Noise floor, stated up front: the same 1,000 questions sent twice through the judge changed 15% of model picks and moved the overall score 0.2 points — anything smaller than ~1.5 points is not a result; costs are "bounded," not "proven."

The four methods:

1. **Reorder for cache** — TURN ON BY DEFAULT. Without reordering 0.1% reused; with reordering 82.2%; speed when busy 4.5 s → 2.7 s; quality cost: none possible (changes no words); costs us 2.5 ms. Detail — under load: one-at-a-time ours 82.2% / 2.76 s vs unordered 0.1% / 2.83 s; 8-at-once ours 79.9% / 2.68 s vs unordered 7.6% / 4.49 s — ours is 40% faster exactly when the gateway is busy.
2. **Summarize old messages** — DO NOT SHIP. Answer quality 73.3 (full) → 32.5 (summarized); 68% less text; −40.8 points; 449/996 answers destroyed. On real conversations where the last question depends on earlier turns, 61% of the damage is the model admitting it cannot answer; the other 39% is confident wrong answers (examples in the source: "Did he want to take him for himself?" Yes→No; "What happened when Champion stepped off the bus?" He vomited → He was beaten; "And LED?" 30,000 hours → 25,000 hours).
3. **Shorten long documents** — TURN ON ABOVE 5,000 WORDS. Before 6,321 words → after 3,266 (48% less text), quality −0.7 points, costs us 0.6 s. Sweep at 1,000 questions (each wrapped in ~6,000 words of context): reference 6,321 tokens / 63.4 quality; same-twice (noise) 6,321 / 63.3 (−0.2); our setting 3,266 / 62.7 (−0.7); gentle 4,875 / 62.1 (−1.3); medium 3,154 / 61.7 (−1.8); aggressive 1,797 / 62.1 (−1.4). Shortening more does not cost more in any clean way; all sit within the judge's own wobble — half the text cut, quality difference too small to pin down.
4. **Reword the question** — LEAVE OFF. Discarded its own work 86% of the time (safety check working correctly on questions with nothing to clarify); before 6,321 words → after 6,315; nothing saved; wastes 1.3 s.

Corrections — three things an earlier round said that were wrong (quoted in substance from the source):

1. We recommended turning on chat summarizing. That was badly wrong. We reported +0.9 points and called it free; the test conversations were built from unrelated questions. On real dependent conversations it costs −40.8 points and destroys 449/996 answers. The flaw was written down as a caveat and left unfixed for two rounds.
2. We said document shortening was free. It is not. At 100 questions it measured exactly zero cost; at 1,000 it costs ~0.7 points and all five settings came out negative. Five out of five pointing the same way is not what randomness looks like.
3. We overstated how much shortening changed the judge's choices. The 2–7% baseline came from short questions and was applied to long ones; on long questions the judge changes 15% of its own picks unprompted. The real effect was 1–7%, not 10–16%.

Limits: long documents and chats are synthetic (built from unrelated text — the easy case; treat −75% and −48% as best cases); document shortening measured which model gets picked, not final answer quality (the same gap that hid the summarizer's failure for two rounds); only the summarizer has been tested on realistic material.

13. PCM scenario test: four methods × twelve request kinds

Source: `sources/13-pcm-scenario-results.pdf` (3,464 requests per method; answered by GPT-5.4-Mini; graded against each dataset's own answer key — no AI judge).

Method. Each real question with a known correct answer is sent twice — as is, and after the method changed it — both answered by GPT-5.4-Mini, both graded. Every request carries the same ~1,100-token gateway policy as in production. Twelve kinds of request (NQ-Open 300 / score 49; MMLU 300 / 81; GSM8K 300 / 94; HumanEval 164 / 93; IFEval 300 / 86; SQuAD 300 / 89; HotpotQA 300 / 74; QuALITY 300 / 78; CoQA 300 / 76; CoQA+NQ-Open 300 / 33; Global-MMLU fa 300 / 69; PersianQA 300 / 76). Two settings per method: shipped (with safety rules) and safety-rules-off. "Not used" = the safety rules correctly left that kind alone. Same question asked twice already moves a score by up to ±6 points; only clearly bigger changes are called real.

Verdicts.

- **Reorder for cache — TURN ON.** 46–99% of each prompt can come from the cache (instead of 0%). Answers better in 5 of 12 kinds (up to +6.0), worse in none. The only column with no red at all.
- **Summarize old chat — KEEP OFF.** Refers-back chats: 76 → 32 (−43.3 points, 145 of 300 answers lost). Topic-change chats: 33 → 43 (+10.1, 36 gained). Both cut chat tokens by the same 68%. The gateway cannot tell which case it faces. Real failure examples: the summary kept the story (a man threw shoes at a president) but not that he said about himself — answer "No." became "unknown"; "Who caught her rapturously?" — Patsy became Mr. Watson (wrong with confidence); after removing a long unrelated Donald Duck chat, "who won the mens figure skating in 2018 olympics" went from "I don't know" to the correct "Yuzuru Hanyu."
- **Rewrite the question — KEEP OFF.** Re-ran the first test (small model Qwen2.5-3B) with GPT-5.4-Mini doing the rewriting — same questions, instruction, and safety check. Still helped none of the 12 kinds; 2 got worse (multiple choice −3.7, chat new-topic −6.0). What the strong model fixed (safety off, small vs strong): strict-format −49.7 → −9.7; long-doc −23.0 → −0.3; multiple choice −18.0 → −3.3; chat-refers-back −13.4 → −2.0. What it didn't: any positive column. Failure mode: harmless polishing ("who played the dad in a christmas story" → "Who played the dad in A Christmas Story?") and the answer became "I don't know" — many lost answers became "I don't know."

- **Remove unimportant words — ONLY LONG DOCUMENTS.** Shipped (5,000+ tokens only): −54% tokens on long documents for −4.3 points (error margin reaches +0.3, so it may be less). Safety rules off: scores fell 24 to 51 points everywhere (short fact −30.5; multiple choice −41.7; math −49.3; code −29.3; strict format −35.0; one passage −43.5; 10 docs −51.1; long doc −46.3; chat refers-back −35.0; chat new-topic −23.5; Persian multiple choice −29.3; Persian passage −44.2). Real failure: "who wrote were going on a bear hunt" → "wrote bear hunt" → answer "Bear Hunt" instead of "Michael Rosen"; the Persian question lost "who else" and "besides."

Bug found and fixed along the way: if the gateway policy merely mentions the personal-data placeholder format `<PII:TYPE:N>`, the safety check mistook it for a damaged placeholder and rejected every method on every request — all four methods would have silently switched themselves off in production. Fixed, with two new tests.

Limits: one answering model (GPT-5.4-Mini); dataset questions already well written (rewriting might do better on genuinely messy prompts; not tested); ~300 questions per kind so changes smaller than ~3 points are invisible; cache share calculated from prompt text (an earlier live test on another model matched this kind of calculation at 82% measured); not tested: tool-calling agents, images, very long chats.

14. Cache paraphrase benchmark

Source: `sources/14-cache-paraphrase-benchmark.pdf` (9,917 labelled question pairs; 2 languages; run twice — guard off and on — plus a six-point sweep of the similarity dial; 30 minutes of compute, no model calls; numbers parsed from the run's own output file).

Design. Five-layer cache in front of a model; replay pairs humans labelled "same question" / "different question." Cache starts empty per pair (store A, ask B); answering from memory on a "same" pair is right, on a "different" pair is a person handed a reply to a question they never asked. Sets: MSRP English (5,301 pairs, 67% same) and ParsiNLU Persian (4,616 pairs, 40% same) — both deliberately adversarial (non-matching pairs use almost as many of the same words as matching ones). Layer ladder: exact → keyword → meaning → examples → model; each step costs more; the guard sits across the middle two. Gap smaller than ~±3.9 points is inside chance.

Verdicts.

- **Exact match — TURN ON.** 0 wrong answers; fires only on letter-for-letter identity after tidying; free and safe in both languages.
- **Keyword match — ENGLISH ONLY.** 6% wrong in English (one in sixteen — tolerable), 33% wrong in Persian (one in three — not).
- **Meaning match — NOT SAFE IN PERSIAN.** 15% wrong in English (strongest layer there) vs 63% wrong in Persian — nearly two of every three Persian reuses belonged to a different question; worse than not caching at all.
- **The guard — PERSIAN ONLY.** Right answers given up per wrong one stopped: 0.88:1 in Persian (fair trade) vs 6.5:1 in English (throws away 6.5 good answers to stop one bad one).

The dial (how alike two questions must look before reuse), share-right / share-caught per setting: English 0.60: 77% / 42%; 0.70: 81% / 36%; 0.80: 86% / 24%; 0.85: 88% / 17%; 0.90: 90% / 10%; 0.95: 95% / 4%. Persian 0.60: 36% / 41%; 0.70: 37% / 41%; 0.80: 40% / 37%; 0.85: 44% / 31%; 0.90: 52% / 22%; 0.95: 70% / 8%. Persian share-right never clears the bar (≥85% right at ≥30% caught) at any setting — there is no blue cell in the Persian columns; no amount of tuning changes it.

What the guard actually does: it does not make reuse more accurate, it makes reuse rarer. It deletes what two prompts share (shared preambles dominate any similarity score) and judges the leftover. Counts: English — 863 right answers given up to stop 133 wrong ones (reuse fell 1,405 → 409); Persian — 1,151 given up, 1,309 stopped (reuse fell 3,229 → 769). Accuracy gains sit inside noise (+2.0 points English vs ±3.4 margin; +3.8 Persian vs ±3.9). Worked examples: "What is the capital of France?" vs "…of Iran?" (shared text 83%,

differing parts match 0% — blocked, correct answer Tehran); "Convert 100 kilometres to miles" vs "250" (blocked on different numbers — 62.1 vs 155.3 miles); "How do I restart a Docker container safely?" vs "How can I reboot a containerized application without downtime?" (11% shared — reused, correct); the same three patterns hold in Persian.

Layer-level wrongness (guard off): English keyword 836 right / −150 wrong; English meaning 393 right / −26 wrong (15% wrong); Persian keyword 790 right / −392 wrong (33%); Persian meaning 756 right / −1,288 wrong (63%) — one layer is responsible for almost all the damage.

The language gap corrected: English reuse right 87% vs Persian 48% — a 40-point gap at face value, but blind reuse already scores 67% vs 40% from the data mix. Points added on top of luck: English +20.4 (guard off) / +22.4 (on); Persian +8.2 / +12.0. Corrected gap: 12 points — Persian adds ~40% as much as English.

Recommendations (from the source): (1) In Persian, switch meaning-match off, keep exact + keyword (the meaning layer was wrong 63% — 1,288 of 2,044 reuses; strictest setting still only 70% right at 8% caught). (2) In English, set the dial by what a wrong answer costs and leave the guard off: 0.85 → 88% right at 17% caught; 0.95 → 95% at 4%. (3) Turn the guard on where shared templates are common or reuse is already unreliable — a filter earns its cost only when what it filters is mostly bad. (4) Measure on your own traffic: this benchmark is built entirely from look-alike pairs; real traffic repeats itself, where exact matching does the work for free.

Found along the way: the meaning-match layer was quietly using an English-only model — no Persian word pieces at all, so unrelated Persian sentences mapped to almost the same point. The model is now configurable and defaults to a multilingual one, and the guard was added as a second, language-independent line of defence. The code is pinned by 129 automated tests.

Limits: both sets adversarial by construction (true hit rate on ordinary traffic will be far higher); the two sets' accuracy figures are not directly comparable without the luck correction; only one sentence model tested (the Persian result may belong to that model rather than to Persian); 1 Persian pair labelled "different" while both sides were identical was detected and excluded, others may remain; speed and money not measured here.

15. Live cache + guard trial, and cascade micro-study

Sources: `sources/16-live-cache-guard-results.json` (preset default, guard on, GPT-4-1106-preview behind the gateway, 5,000 prompts, cached:sbert embedder) and `sources/15-cascade-vs-mini-only-comparison.md` (20 questions).

Live trial. Cache: 5 hits (hit rate 0.001), of which 1 semantic and 4 keyword; 2 false hits (40% of hits — on look-alike-heavy benchmark traffic this is expected; real traffic repeats exactly, where exact matching is safe). Cost: $16.1211 without cache vs $16.1072 with — $0.014 saved (0.09%). Quality: 0.7851 without vs 0.7846 with (−0.0004). The honest reading: on this traffic mix the near-zero hit rate is the story, not a quality problem — it directly supports the architecture's decision to re-baseline the ">30% hit rate" Stage-2 gate on real logged traffic rather than trust the literature anchor.

Cascade vs mini-only (20 questions): cascade (nano → mini) 17/20 correct (85.0%) at 1,723 total completion tokens, escalating 10/20 (50%) to mini; mini-only 18/20 (90.0%) at 1,391 tokens. Accuracy delta −5.0 points; token "savings" −23.9% fewer (i.e., the cascade used more tokens: nano's 1,212 wasted tokens on escalated questions). At this scale the accuracy interval spans zero (n=20), but the direction is consistent with the five-study synthesis: a small-model-first cascade without a reliable escalation signal pays tokens and accuracy.

16. Summarization & speculative decoding survey

Source: `sources/17-summarization-and-speculative-decoding-survey/` (LaTeX). A literature survey that framed the later PCM work: online summarization as a systems level optimization (context compression; conversation memory compression; multi-model response aggregation; consensus and disagreement analysis; hierarchical summarization; adaptive summarization policies), and online verification (classical speculative decoding — draft model proposes, target verifies in one forward pass, ~2–3× speedup preserving

the output distribution; self-speculative decoding via Draft & Verify layer skipping; SpecInfer tree-based multi-branch verification; adaptive speculation policies; verification beyond decoding — fact/consistency/ hallucination/safety/tool/ citation/code checks; selective verification of numerical, factual, referenced, code, and safety-sensitive regions; verification as a post-routing escalation stage). Proposed research directions include query-aware compression, budget-aware summarization policies, consensus-guided aggregation, and joint optimization of routing, verification, and escalation.

Part V — Consolidated findings and recommendations

17. What five months established

1. **Anchor routing is the production choice.** 78.67% at $0.002751/query on RouterBench held-out (+0.57 pp [0.22, 0.91] over the GPT-4 floor, 16% cheaper), 0.64 ms on CPU (24× faster than Router-BERT at statistically indistinguishable quality; seven budget-level head-to-heads all ties). On the ArvanCloud pool it beat all seven models (93.3% / 637 tokens vs Sol's 92.7% / 750). Its limits are now measured too: with only 60 training prompts (8 anchors) it loses to weighted coin flips (−0.62 pp, winning at 7% of operating points) — anchor learns prompt-easiness, and needs data density (29,185 training prompts gave +5.16 pp over the convex hull at 98% of operating points).
2. **Cascades substitute for a poor router, not enhance a good one.** 23 head-to-heads at matched spend: standalone anchor won every one. On ArvanCloud, anchor+cascade buys +1.1 points for 2.4× tokens with an interval spanning zero. The escalation signal is the leverage point: a perfect wrongness signal would gain +10.84 pp while cutting cost to 0.87×; the agreement signal escalates 38% of correct answers. The ETH-style cascade never escalated in practice (83.3%, 428 tokens — it behaved like a cheap-model router). Token-logprob cascading was structurally unavailable (1 of 9 models returns logprobs).
3. **Compression is conditional, and the conditions are now measured.** Reordering is free and unconditionally good (0.1% → 82.2% reuse; 40% faster at 8-way concurrency). Word removal pays only above 5,000 tokens (−48% text for −0.7 points; below the gate it slows requests and, without safety rules, costs 24–51 points). Summarization of chat history is unsafe when questions depend on history (−40.8 points, 449/996 destroyed; 61% "I don't know," 39% confidently wrong). Rewriting helps nothing (0 of 12 kinds better even with a strong rewriter) and rejects its own output 86% of the time.
4. **The cache must be governed per language.** Persian meaning-match is 63% wrong; the guard trades 0.88:1 in Persian (keep) vs 6.5:1 in English (skip). The corrected language gap is 12 points of cache skill, not 40. The live 5,000-prompt trial (hit rate 0.001) confirms that hit-rate gates must be re-baselined on real traffic.
5. **The evaluation process corrected itself, on the record.** Three earlier claims reversed under proper testing (summarization "free" → −40.8; shortening "free" → −0.7; judge-pick changes 10–16% → 1–7%). The August study's "0.00 pp accuracy cost" was honestly bounded rather than claimed as improvement. Bugs found by benchmarks that unit tests missed: the prose-matching code detector (28/100 mislabelled), the LLMLingua duplicate-token crash, the `<PII:TYPE:N>` policy-mention self-disable that would have switched all four methods off in production, the English-only embedding model in the cache.
6. **The architecture makes the budget contract structural.** Ledger reservations before dispatch, watchdog at deadline − t_floor, four distinct client-visible outcomes, raced (never blocking) semantic cache, output guard on every answer, and a feature-flag ladder whose enable conditions are measurements. Nothing in the benchmark results contradicts the design; several results (anchor's CPU speed, the cascade caveats, the cache hit-rate reality) directly de-risked its choices.

18. Recommendations

1. **Ship anchor routing as the default**, rules as the permanent floor, with the Arch-Router-1.5B slot optional behind its 75 ms timeout gate. Keep router overhead p99 ≤ 25 ms as the design budget (anchor: 0.64 ms).
2. **Turn on canonicalization/reordering everywhere.** It is the only intervention that is free, unconditionally quality-safe, and pays under load.
3. **Keep compressor.min_tokens = 5000** and do not lower it; both ablation studies and the scenario test independently support the gate. Re-run on real long-context traffic before quoting −48% externally (current evidence is synthetic-best-case).
4. **Keep summarization off** until topic-change detection exists (it is the stated precondition for safety); if revisited, try the four fixes proposed in source 12 (never summarize a document-bearing turn, keep more recent turns verbatim, summarize with the latest question in view, skip summarization on references-back).
5. **Keep rewriting off** for this traffic class.
6. **Cache policy per language:** Persian exact + keyword with guard on; English dial 0.85 or 0.95 with guard off; re-baseline every hit-rate gate on logged traffic; keep per-tenant isolation and the verified-write gate (CacheAttack's 80–88% hijack rates against shared production caches).
7. **Cascades:** not default. Pursue escalation specificity (target sensitivity ~0.75, specificity 0.62 → 0.75) rather than more cascade machinery.
8. **Close the two measurement gaps before external claims:** real retrieved documents for compression; real traffic hit rates for the cache. Both were flagged as best-case in their sources.

19. Limits of the whole five-month evidence base

- **Synthetic long-context.** All long-document and chat results are built from unrelated RouterBench questions — the easy case. −47.6%/−48% and −81.4% are upper bounds.
- **Routing-decision vs answer quality.** The PCM studies measure which model gets picked, not the final answer (except the scenario test, which grades real answers with one model, GPT-5.4-Mini). A weaker answering model may be hurt more.
- **Sample sizes.** 90 held-out ArvanCloud questions ≈ ±6 pp variance; n=100 PCM arms give ~±4 pp CIs; n=20 in the cascade micro-study proves nothing on its own; ~300 per request kind in the scenario test cannot see <3-point changes.
- **Pricing.** OpenRouter list prices (2026-09-22) stood in for ArvanCloud tariffs; token counts are a 4.9× misleading proxy for the real 86× price span.
- **Model coverage.** Logprobs on 1 of 9 evaluated models; several RouterBench models never latency-measured (recorded `measured: false`, never guessed); one answering model in the scenario tests; one sentence model in the cache benchmark.
- **Language.** Persian findings (routing, cache, PII patterns) rest on benchmarks adapted rather than purpose-built; judge agreement on Persian is unvalidated.